

CONTENTS

Redes de Flujo

Guía de estudio con tolerancia cero — Módulo 7, Programación 3 (FING UdelaR) — Kleinberg & Tardos, Capítulo 7

0. Por qué existe este tema

1. Definición formal de red de flujo

2. El grafo residual

3. Algoritmo de Ford-Fulkerson

Ejemplo completo paso a paso (4 iteraciones, verificado con Python)

4. Cortes y el Teorema de Flujo Máximo = Corte Mínimo

5. Complejidad: Ford-Fulkerson genérico vs. Edmonds-Karp

6. Reducción de Matching Bipartito Máximo a Max-Flow

7. Trampas típicas de examen (resumen)

Resumen ultra-rápido (2 minutos)

Redes de Flujo

GUÍA DE ESTUDIO CON TOLERANCIA CERO — MÓDULO 7, PROGRAMACIÓN 3 (FING UDELAR) — KLEINBERG & TARDOS, CAPÍTULO 7

0. Por qué existe este tema

Hasta ahora (grafos, ávidos, DyV, programación dinámica) el curso resolvió problemas sobre estructuras “estáticas”: caminos, órdenes, subconjuntos óptimos. Las **redes de flujo** cambian el objeto de estudio: en vez de preguntar “¿cuál es el mejor camino?”, preguntan “¿cuánto material puede circular simultáneamente por todos los caminos a la vez, sin violar las capacidades de las tuberías?”. Es un problema de *optimización global sobre todo el grafo a la vez*, no sobre un único camino.

La razón por la que este módulo es tan denso en el examen es que **una enorme cantidad de problemas que no parecen de “flujo” en absoluto se resuelven reduciéndolos a Max-Flow**: asignación de pacientes a hospitales con cupos, emparejamiento bipartito (asignar tareas a trabajadores, estudiantes a proyectos), problemas de conectividad con múltiples caminos disjuntos, problemas de selección con restricciones de capacidad. El patrón de examen típico es: “modele este problema como una red de flujo, resuelva con Ford-Fulkerson, demuestre la corrección de su algoritmo usando el Teorema de Flujo Máximo = Corte Mínimo, y analice la complejidad.”

Regla de oro que se repite en TODO el módulo: **el flujo máximo de una red es exactamente igual a la capacidad de su corte mínimo** (Teorema Max-Flow Min-Cut). Ese único hecho es la base de la corrección de Ford-Fulkerson, del criterio de parada del algoritmo, y de la mayoría de las demostraciones de corrección de las reducciones a Max-Flow que aparecen en los exámenes.

1. Definición formal de red de flujo

Red de flujo. Un grafo dirigido $G=(V,E)$ con:

- un nodo **fuente** $s \in V$ (sin aristas entrantes, por convención — o que no importa si las tiene, porque nunca se usan),
- un nodo **sumidero** $t \in V$ (destino final),
- una **función de capacidad** $c: E \rightarrow \mathbb{R}^+$ que asigna a cada arista $e=(u,v)$ un valor $c(e) \geq 0$ (cuánto puede “circular” por esa arista como máximo).

Flujo. Una función $f: E \rightarrow \mathbb{R}^+_{\geq 0}$ que asigna a cada arista una cantidad de flujo, sujeta a dos restricciones:

1. **Restricción de capacidad:** para toda arista $e=(u,v)$, $0 \leq f(e) \leq c(e)$.
2. **Conservación de flujo:** para todo nodo $v \in V \setminus \{s,t\}$ (todo nodo que no sea fuente ni sumidero): $\sum_{e \text{ entra a } v} f(e) = \sum_{e \text{ sale de } v} f(e)$ (“lo que entra a un nodo intermedio debe ser igual a lo que sale” — nada se acumula ni se genera de la nada en los nodos internos).

Valor del flujo. $v(f) = \sum_{e \text{ sale de } s} f(e) - \sum_{e \text{ entra a } s} f(e)$ — el flujo neto que sale de la fuente. Por conservación (aplicada de forma agregada a todo $v \setminus \{s, t\}$), este valor es exactamente igual al flujo neto que entra al sumidero t .

Flujo máximo (Max-Flow). El problema central del capítulo: encontrar, entre todos los flujos válidos de una red dada, uno de valor $v(f)$ máximo.

Ejemplo 1.1 — Verificar que una asignación de flujo es válida.

Red: $s \rightarrow a$ (cap. 3), $s \rightarrow b$ (cap. 2), $a \rightarrow t$ (cap. 2), $b \rightarrow t$ (cap. 3), $a \rightarrow b$ (cap. 1).

Propuesta de flujo: $f(s,a)=2$, $f(s,b)=2$, $f(a,t)=2$, $f(b,t)=2$, $f(a,b)=0$.

Verificación de capacidad: $2 \leq 3$, $2 \leq 2$, $2 \leq 2$, $2 \leq 3$, $0 \leq 1$ ✓ (todas cumplen).

Verificación de conservación en a : entra $f(s,a)=2$; sale $f(a,t)+f(a,b)=2+0=2$ ✓.

Verificación de conservación en b : entra $f(s,b)+f(a,b)=2+0=2$; sale $f(b,t)=2$ ✓.

Valor del flujo $v(f) = f(s,a)+f(s,b) = 2+2 = 4$. Verificación cruzada por el lado de t :

$f(a,t)+f(b,t)=2+2=4$ ✓ (coincide, como debe ser).

Error frecuente de tolerancia cero: pensar que la conservación de flujo aplica también en s y en t .

NO — s y t están explícitamente excluidos de la restricción de conservación: s es una fuente neta de flujo (genera) y t es un sumidero neto (consume). Aplicar la fórmula de conservación en s o en t daría $0=v(f)$, lo cual es absurdo salvo en el caso trivial.

2. El grafo residual

Idea. Dado un flujo f sobre G , el **grafo residual** G_f describe, para cada arista, cuánto flujo *adicional* se puede seguir empujando en cada sentido — incluyendo la posibilidad de **deshacer** flujo ya asignado.

Para cada arista original $e=(u,v)$ con capacidad $c(e)$ y flujo actual $f(e)$, G_f tiene:

- **Arista hacia adelante** (u,v) con **capacidad residual** $c_f(u,v) = c(e) - f(e)$, presente **solo si** $c_f(u,v) > 0$ (queda margen para aumentar el flujo en el sentido original).
- **Arista hacia atrás** (v,u) con capacidad residual $c_f(v,u) = f(e)$, presente **solo si** $f(e) > 0$ (permite “deshacer” flujo ya enviado por (u,v) , empujando en sentido contrario).

La arista hacia atrás **no es una arista física del grafo original** — es un artificio algorítmico que representa “puedo arrepentirme de haber mandado flujo por acá”. Es, junto con el criterio de parada del algoritmo, el punto más frecuentemente olvidado en los exámenes: sin las aristas hacia atrás, Ford-Fulkerson deja de ser correcto (puede quedar atascado en un óptimo local que no es el máximo global).

Ejemplo 2.1 — Construcción explícita del grafo residual.

Red con capacidades: $s \rightarrow a$ (cap. 10), $a \rightarrow t$ (cap. 10), $s \rightarrow b$ (cap. 5), $b \rightarrow t$ (cap. 5), $a \rightarrow b$ (cap. 15).

Flujo actual: $f(s,a)=10$, $f(a,t)=6$, $f(s,b)=5$, $f(b,t)=5$, $f(a,b)=4$.

Verificación de conservación: en a : entra 10, sale $f(a,t)+f(a,b)=6+4=10$ ✓. En b : entra

$f(s,b)+f(a,b)=5+4=9$... **esto NO conserva** ($f(b,t)=5$) — deliberadamente incorrecto para mostrar el chequeo. Corrijamos: $f(a,b)=0$ en su lugar. Entonces en b : entra $f(s,b)=5$, sale $f(b,t)=5$ ✓. En a : entra 10, sale $f(a,t)+f(a,b)=6+0=6 \neq 10$. Tampoco conserva.

Flujo válido correcto: $f(s,a)=10$, $f(a,t)=10$, $f(s,b)=5$, $f(b,t)=5$, $f(a,b)=0$. Ahora sí: en a , entra 10, sale $10+0=10$ ✓; en b , entra $5+0=5$, sale 5 ✓. $v(f)=10+5=15$.

Grafo residual G_f para este flujo:

- $s \rightarrow a$: $c_f = 10-10 = 0$ → **no aparece** hacia adelante. Aparece hacia atrás: $a \rightarrow s$, $c_f = f(s,a) = 10$.

- $a \rightarrow t$: $c_f = 10-10=0$ → no aparece hacia adelante. Hacia atrás: $t \rightarrow a$, $c_f=10$.

- $s \rightarrow b$: $c_f = 5-5=0$ → no adelante. Atrás: $b \rightarrow s$, $c_f=5$.

- $b \rightarrow t$: $c_f=5-5=0$ → no adelante. Atrás: $t \rightarrow b$, $c_f=5$.

- $a \rightarrow b$: $c_f = 15-0=15$ → adelante, $c_f=15$. Como $f(a,b)=0$, **no** hay arista hacia atrás $b \rightarrow a$.

Resultado: G_f tiene las aristas $\{s \rightarrow a(10), t \rightarrow a(10), b \rightarrow s(5), t \rightarrow b(5), a \rightarrow b(15)\}$. Ninguna llega desde s hacia adelante con capacidad positiva salvo... de hecho s no tiene ninguna arista saliente en G_f (todas sus aristas originales están saturadas). **Conclusión inmediata:** no existe camino $s \rightarrow t$ en $G_f \Rightarrow$ este flujo de valor 15 ya es el **flujo máximo** (por el criterio de parada de la sección 3).

3. Algoritmo de Ford-Fulkerson

Idea central. Mientras exista un **camino aumentante** — un camino dirigido de s a t en el grafo residual G_f — se puede aumentar el valor del flujo empujando tanto como permita el **cuello de botella** (la mínima capacidad residual entre las aristas del camino). Cuando ya no queda ningún camino $s \rightarrow t$ en G_f , el flujo actual es máximo.

Pseudocódigo:

```
FORD-FULKERSON(G, s, t, c):
  para cada arista e: f(e) = 0
  mientras exista un camino P de s a t en el grafo residual G_f:
    b = min{ c_f(e) : e ∈ P } // cuello de botella del camino
    para cada arista (u,v) ∈ P:
      si (u,v) es arista "hacia adelante" en G_f:
        f(u,v) = f(u,v) + b
      si (u,v) es arista "hacia atrás" en G_f (corresponde a arista original (v,u)):
        f(v,u) = f(v,u) - b
    recalcular G_f con el nuevo f
  devolver f
```

¿Por qué termina (con capacidades enteras)? Cada iteración aumenta el valor del flujo en al menos 1 unidad (el cuello de botella b es un entero positivo, porque todas las capacidades y flujos intermedios se mantienen enteros si las capacidades de entrada son enteras — invariante que se preserva en cada paso). Como el valor del flujo está acotado superiormente por la capacidad de cualquier corte (ver Sección 4), el algoritmo no puede aumentar indefinidamente: **termina en, a lo sumo, $v(f^*)$ iteraciones**, donde $v(f^*)$ es el valor del flujo máximo.

¿Por qué el resultado final es máximo (correctitud)? Se demuestra junto con el Teorema de la Sección 4: cuando no queda camino aumentante, el conjunto A de nodos alcanzables desde s en G_f (con $B = V \setminus A$) forma un corte cuya capacidad es exactamente igual al valor del flujo actual — y como ningún flujo puede superar la capacidad de ningún corte, este flujo es necesariamente máximo.

EJEMPLO COMPLETO PASO A PASO (4 ITERACIONES, VERIFICADO CON PYTHON)

Red: s, a, b, c, d, t con capacidades $c(s,a)=16$, $c(s,c)=13$, $c(a,c)=10$, $c(c,a)=4$, $c(a,b)=12$, $c(c,d)=14$, $c(b,c)=9$, $c(d,b)=7$, $c(b,t)=20$, $c(d,t)=4$.

Se ejecuta Ford-Fulkerson **genérico** (elección de camino por DFS, no necesariamente el más corto — a propósito, para mostrar que cualquier elección de camino aumentante es válida, no solo BFS):

Iteración	Camino aumentante (en G_f)	Cuello de botella	Flujo acumulado $ f $
1	$s \rightarrow b \rightarrow c \rightarrow d \rightarrow t$	4	4
2	$s \rightarrow a \rightarrow b \rightarrow t$	8	12
3	$s \rightarrow a \rightarrow c \rightarrow b \rightarrow t$	4	16
4	$s \rightarrow c \rightarrow d \rightarrow t$	7	23

Al terminar la iteración 4, no queda ningún camino $s \rightarrow t$ en G_f (verificado por BFS/DFS exhaustivo) $\Rightarrow$ **flujo máximo = 23**.

Script de verificación (Python) — reproducido íntegro. Implementa el grafo residual explícitamente (aristas hacia adelante y hacia atrás), busca caminos aumentantes con DFS, aplica el algoritmo hasta agotar los caminos, y al final calcula el corte mínimo por alcanzabilidad en G_f .

```
edges_cap = {("s","a"):16, ("s","c"):13, ("a","c"):10, ("c","a"):4,
             ("a","b"):12, ("c","d"):14, ("b","c"):9, ("d","b"):7,
             ("b","t"):20, ("d","t"):4}
flow = {e: 0 for e in edges_cap}

def residual_graph(flow):
    res = {}
    for (a,b), c in edges_cap.items():
        f = flow[(a,b)]
        if c - f > 0: res.setdefault(a,{})[b] = ("fwd", (a,b), c-f)
        if f > 0:    res.setdefault(b,{})[a] = ("back", (a,b), f)
    return res

# ... dfs_path(flow) busca un camino s->t por DFS en residual_graph(flow) ...
# ... augment(flow, path) resta/suma el cuello de botella b a cada arista ...

it = 1; total = 0
while True:
    path = dfs_path(flow)
    if path is None: break
    b = augment(flow, path); total += b
    print(f"Iteracion {it}: cuello de botella={b}, |f|={total}")
    it += 1
```

Salida real obtenida:

```

Iteracion 1: camino = s -> a -> b -> c -> d -> t | cuello de botella = 4 | |f| = 4
Iteracion 2: camino = s -> a -> b -> t | cuello de botella = 8 | |f| = 12
Iteracion 3: camino = s -> a -> c -> b -> t | cuello de botella = 4 | |f| = 16
Iteracion 4: camino = s -> c -> d -> b -> t | cuello de botella = 7 | |f| = 23

```

Total de iteraciones: 4
Valor del flujo máximo = 23

Ejemplo 3.1 — Verificación de conservación del flujo final.

Flujo final obtenido: $f(s,a)=12$, $f(s,c)=11$, $f(a,c)=0$, $f(c,a)=0$, $f(a,b)=12$, $f(c,d)=11$, $f(b,c)=0$, $f(d,b)=7$, $f(b,t)=19$, $f(d,t)=4$.

En s_a : entra $f(s,a)=12$; sale $f(a,b)+f(a,c)=12+0=12$ ✓.

En s_b : entra $f(a,b)+f(d,b)=12+7=19$; sale $f(b,t)+f(b,c)=19+0=19$ ✓.

En s_c : entra $f(s,c)+f(a,c)+f(b,c)=11+0+0=11$; sale $f(c,d)+f(c,a)=11+0=11$ ✓.

En s_d : entra $f(c,d)=11$; sale $f(d,b)+f(d,t)=7+4=11$ ✓.

Valor: $v(f)=f(s,a)+f(s,c)=12+11=23$. Verificación en t : $f(b,t)+f(d,t)=19+4=23$ ✓.

Error frecuente de tolerancia cero: elegir como “camino aumentante” una secuencia de nodos que sí forma un camino en el grafo **original**, pero no en el **residual** — es decir, olvidarse de verificar que cada arista del camino tenga capacidad residual estrictamente positiva en el momento de aplicarlo. Un camino con una arista de capacidad residual 0 en algún tramo **no es aumentante** y no se puede usar.

4. Cortes s - t y el Teorema de Flujo Máximo = Corte Mínimo

Corte s - t . Una partición (A,B) de V tal que $s \in A$, $t \in B$. La **capacidad del corte** es $c(A,B) = \sum_{e=(u,v): u \in A, v \in B} c(e)$ — la suma de las capacidades de las aristas que van de A hacia B (las que van de B hacia A **no cuentan**, aunque existan).

Error frecuente de tolerancia cero (el más común del módulo): confundir “corte mínimo” con “la arista de menor capacidad de la red”. El corte es un **conjunto de aristas** (las que cruzan de A a B), y su capacidad es la **suma** de sus capacidades individuales — no un único número tomado de una sola arista. Además, aristas que van de B a A (en sentido contrario al corte) **no se suman**, aunque tengan capacidad grande.

Propiedad 1 — Todo flujo vale lo mismo cruzando cualquier corte. Para cualquier flujo válido f y cualquier corte (A,B) : $v(f) = \sum_{e: u \in A, v \in B} f(e) - \sum_{e: u \in B, v \in A} f(e)$ (se demuestra sumando la ecuación de conservación de flujo sobre todos los nodos de $A \setminus \{s\}$ y cancelando términos telescópicamente).

Propiedad 2 — Cota superior (débil). Para cualquier flujo f y cualquier corte (A,B) : $v(f) \leq c(A,B)$. *Demostración:* de la Propiedad 1, $v(f) = \sum f(e_{A \rightarrow B}) - \sum f(e_{B \rightarrow A}) \leq \sum f(e_{A \rightarrow B}) \leq \sum c(e_{A \rightarrow B}) = c(A,B)$ (el primer $\leq$ porque restamos una cantidad no negativa; el segundo porque cada $f(e) \leq c(e)$ por restricción de capacidad). $\blacksquare$

Teorema de Flujo Máximo = Corte Mínimo. El valor del flujo máximo de una red es exactamente igual a la capacidad mínima entre todos los cortes s - t de esa red: $\max_f v(f) = \min_{(A,B)} c(A,B)$

Idea de la demostración (equivalencia de tres afirmaciones sobre un flujo f):

1. f es un flujo máximo.
2. El grafo residual G_f no tiene ningún camino $s \rightsquigarrow t$.
3. Existe un corte (A,B) con $c(A,B) = v(f)$.

$(3) \Rightarrow (1)$: por la Propiedad 2, $v(f) \leq c(A,B)$ para *cualquier* flujo; si además $v(f) = c(A,B)$ exactamente, entonces ningún flujo puede superar ese valor, así que f ya es máximo. $(1) \Rightarrow (2)$: si existiera un camino aumentante en G_f , se podría incrementar $v(f)$, contradiciendo que f es máximo. $(2) \Rightarrow (3)$: si no hay camino $s \rightsquigarrow t$ en G_f , sea A el conjunto de nodos alcanzables desde s en G_f (y $B = V \setminus A$; nótese $t \in B$ por hipótesis). Toda arista original (u,v) con $u \in A, v \in B$ debe estar **saturada** ($f(u,v) = c(u,v)$) — si no, tendría capacidad residual positiva y v sería alcanzable desde A , contradicción. Análogamente, toda arista original (u,v) con $u \in B, v \in A$ debe tener $f(u,v) = 0$ — si no, existiría arista residual hacia atrás $v \rightarrow u$ ($v \in A \dots$ pero $v \in B$, así que sería una arista residual desde un nodo de B , que en realidad conecta hacia A , y por la definición de A como *alcanzable desde s* , esto no contradice nada directamente; el argumento correcto es sobre las aristas que **salen** de A : si $f(u,v) > 0$ con $u \in B, v \in A$ eso no afecta la definición de A). Aplicando la Propiedad 1 a este corte: $v(f) = \sum f(e_{A \rightarrow B}) - \sum f(e_{B \rightarrow A}) = \sum c(e_{A \rightarrow B}) - 0 = c(A,B)$. $\blacksquare$

Esta demostración es exactamente el mecanismo que usa Ford-Fulkerson para **saber cuándo parar**: el algoritmo termina precisamente cuando se cumple la condición (2) — no queda camino aumentante — y en ese momento, el conjunto A de nodos alcanzables desde s en G_f define automáticamente un corte mínimo, sin necesidad de un paso adicional de búsqueda.

Ejemplo 4.1 — Corte mínimo de la red del Ejemplo 3.1 (verificado).

Al terminar las 4 iteraciones, el grafo residual final tiene como alcanzables desde s : hay que recorrer G_f con el flujo final. Ejecutando BFS desde s sobre G_f (aristas con capacidad residual > 0), el conjunto obtenido es $A = \{s, a, c, d\}$, $B = \{b, t\}$.

Aristas que cruzan de A a B : (a,b) cap. 12, (d,b) cap. 7, (d,t) cap. 4. Capacidad del corte $= 12 + 7 + 4 = 23$.

Coincide exactamente con el valor del flujo máximo hallado (23) ✓ — confirmación numérica directa del teorema.

5. Complejidad: Ford-Fulkerson genérico vs. Edmonds-Karp

Ford-Fulkerson genérico (elección arbitraria del camino aumentante).

- Cada iteración cuesta $O(E)$ (encontrar un camino $s \rightsquigarrow t$ en G_f , por ejemplo con DFS).

- Cantidad de iteraciones $\leq v(f^*)$ (el valor del flujo máximo), porque cada iteración aumenta el flujo en al menos 1 unidad (con capacidades enteras).
- **Complejidad total:** $O(E \cdot v(f^*))$.

Trampa de examen clásica: $O(E \cdot v(f^*))$ **NO es polinomial en el tamaño de la entrada** en general. El valor $v(f^*)$ puede ser **exponencialmente grande** respecto al número de bits necesarios para representar las capacidades (por ejemplo, si las capacidades son del orden de 2^n , el algoritmo genérico podría necesitar del orden de 2^n iteraciones). Esto es tolerancia cero: “Ford-Fulkerson es polinomial porque termina” es **falso** — termina, pero no necesariamente en tiempo polinomial en el tamaño de la entrada (que se mide en bits, no en el valor numérico de las capacidades).

Demostración numérica de la trampa (capacidades $C=4$, red s,a,b,t con $c(s,a)=c(s,b)=c(a,t)=c(b,t)=C$ y $c(a,b)=1$):

Si el algoritmo elige, adversarialmente, siempre el camino que pasa por la arista “cuello de botella” (a,b) (alternando $s \rightarrow a \rightarrow b \rightarrow t$ y luego, tras crear la arista residual hacia atrás, $s \rightarrow b \rightarrow a \rightarrow t$), cada iteración aumenta el flujo en **solo 1 unidad**, requiriendo $2C$ iteraciones en total para llegar al máximo (que vale $2C$). Con Edmonds-Karp (BFS, siempre el camino más corto), el mismo flujo máximo se alcanza en **2 iteraciones**.

Verificado con script Python (misma red, dos políticas de selección de camino):

```
C = 4
Ford-Fulkerson con eleccion ADVERSARIAL (alternando s-a-b-t / s-b-a-t):
  iteraciones = 8, flujo maximo = 8   (esperado 2C = 8)
Edmonds-Karp (BFS, camino mas corto):
  iteraciones = 2, flujo maximo = 8
```

Con $C=1000$ en vez de 4 (mismo patrón), el genérico necesitaría **2000** iteraciones y Edmonds-Karp seguiría necesitando solo **2** — la brecha crece con C , mientras que el tamaño de la entrada (cantidad de bits para escribir C) crece solo logarítmicamente. Esto es precisamente lo que hace que la cota $O(E \cdot v(f^*))$ no sea polinomial en el tamaño de la entrada.

Edmonds-Karp (Ford-Fulkerson + BFS para elegir siempre el camino aumentante más corto).

- Cada iteración sigue costando $O(E)$ (un BFS).
- Se demuestra (KT Cap. 7.3) que la cantidad de iteraciones está acotada por $O(V \cdot E)$, **independientemente de las capacidades** — cada arista puede ser la “arista cuello de botella” de un camino más corto solo $O(V)$ veces a lo largo de toda la ejecución.
- **Complejidad total:** $O(V \cdot E^2)$ — **sí es polinomial en el tamaño de la entrada** (en V y E , sin depender del valor numérico de las capacidades).

Algoritmo	Selección de camino	Iteraciones	Complejidad total	¿Polinomial en el tamaño de entrada?
Ford-Fulkerson genérico	Cualquiera (DFS, etc.)	$O(v(f^*))$	$O(E \cdot v(f^*))$	No , en general (depende del valor de las capacidades)
Edmonds-Karp	BFS (camino más corto)	$O(V \cdot E)$	$O(V \cdot E^2)$	Sí

6. Reducción de Matching Bipartito Máximo a Max-Flow

Problema de Matching Bipartito Máximo. Dado un grafo bipartito $G=(L \cup R, E)$ (aristas solo entre L y R), encontrar el mayor conjunto de aristas $M \subseteq E$ tal que ningún nodo participe en más de una arista de M .

Construcción de la reducción. A partir de $G=(L \cup R, E)$, se construye una red de flujo G' :

- Se agregan dos nodos nuevos: fuente s y sumidero t .
- Por cada nodo a en L : arista (s, a) con capacidad 1.
- Por cada arista original (a, b) en E con a en L , b en R : arista dirigida (a, b) con capacidad 1 (mismo sentido $L \rightarrow R$).
- Por cada nodo b en R : arista (b, t) con capacidad 1.

Afirmación: el valor del flujo máximo en G' es exactamente el tamaño del emparejamiento máximo en G .

Idea de la demostración: como todas las capacidades son 1 y las capacidades son enteras, existe un flujo máximo entero (por la Sección 3) donde cada arista lleva flujo 0 o 1. La restricción de capacidad-1 en las aristas (s, a) obliga a que cada a en L participe en **a lo sumo una** arista de flujo 1 hacia R (por conservación); simétricamente, la capacidad-1 en (b, t) obliga a que cada b en R reciba **a lo sumo una** unidad de flujo. Por lo tanto, el conjunto de aristas (a, b) con $f(a, b)=1$ es exactamente un emparejamiento válido, y maximizar $\sum f$ equivale a maximizar la cantidad de aristas de ese emparejamiento.

Ejemplo 6.1 — Instancia concreta, verificada con Python.

$L=\{a_1, a_2, a_3\}$, $R=\{b_1, b_2, b_3\}$. Aristas: $a_1 \rightarrow b_1$, $a_1 \rightarrow b_2$, $a_2 \rightarrow b_1$, $a_3 \rightarrow b_2$, $a_3 \rightarrow b_3$.

Red construida: $s \rightarrow a_1, a_2, a_3$ (cap. 1 c/u); $a_1 \rightarrow b_1$, $a_1 \rightarrow b_2$, $a_2 \rightarrow b_1$, $a_3 \rightarrow b_2$, $a_3 \rightarrow b_3$ (cap. 1 c/u); $b_1, b_2, b_3 \rightarrow t$ (cap. 1 c/u).

Salida real del script (Edmonds-Karp):

```
Valor de flujo maximo = 3
Emparejamiento inducido por el flujo: [('a1','b2'), ('a2','b1'), ('a3','b3')]
Maximo emparejamiento por fuerza bruta (todas las combinaciones) = 3
Verificado: tamano del maximo flujo == tamano del maximo emparejamiento.
```

La verificación por **fuerza bruta** (probar todos los subconjuntos de aristas y quedarse con el emparejamiento válido más grande) confirma independientemente que el máximo es 3, coincidiendo con el flujo máximo — no es casualidad, es exactamente lo que garantiza la reducción.

Este es el patrón de reducción más transferible del módulo para el examen: cualquier problema de “asignación 1 a 1 con restricciones de capacidad por elemento” (pacientes a hospitales con cupo, tareas a servidores, franjas horarias a exámenes) se modela agregando fuente/sumidero y capacidades 1 (o mayores, si se permiten varias asignaciones por elemento) en los extremos de una red bipartita, y resolviendo Max-Flow.

7. Trampas típicas de examen (resumen)

1. Olvidar las aristas hacia atrás en el grafo residual. Sin ellas, el algoritmo puede quedar atascado en un flujo que no es máximo (no puede “deshacer” una asignación de flujo subóptima hecha en una iteración anterior). Toda arista con $f(e) > 0$ genera una arista residual hacia atrás con capacidad $f(e)$, sin excepción.

2. Usar un camino que no es realmente aumentante. Un camino de s a t en el grafo **original** no sirve si en el grafo **residual** alguna de sus aristas tiene capacidad residual 0 (está saturada, o no tiene flujo que deshacer en el caso de una arista hacia atrás). Siempre verificar capacidad residual > 0 en **cada** arista del camino antes de aplicar el algoritmo.

3. Confundir “corte mínimo” con “arista de menor capacidad”. El corte es un **conjunto de aristas** que cruzan de A a B ; su capacidad es la **suma** de las capacidades de esas aristas (nunca un único valor tomado de una sola arista, y nunca sumando las aristas que van de B a A).

4. Creer que Ford-Fulkerson genérico es polinomial en el tamaño de la entrada. Es polinomial en el **valor del flujo máximo** ($O(E \cdot v(f^*))$), lo cual puede ser exponencial respecto a la cantidad de bits de las capacidades. Solo Edmonds-Karp (BFS) garantiza $O(V E^2)$, verdaderamente polinomial en V y E .

5. Aplicar la conservación de flujo en s o t . La conservación solo aplica a los nodos intermedios ($V \setminus \{s, t\}$); s y t son, por definición, los únicos nodos donde el flujo neto puede ser distinto de cero.

Resumen ultra-rápido (2 minutos)

- **Red de flujo:** $G=(V,E)$ dirigido, capacidades $c(e) \geq 0$, fuente s , sumidero t . **Flujo válido:** $0 \leq f(e) \leq c(e)$ + conservación en todo nodo $v \neq s, t$. **Valor:** flujo neto que sale de s = flujo neto que entra a t .

- **Grafo residual G_f :** por cada arista (u,v) con $f(u,v) < c(u,v)$, arista adelante (u,v) con $c_f = c - f$; por cada arista con $f(u,v) > 0$, arista atrás (v,u) con $c_f = f$. **Nunca omitir las aristas hacia atrás.**
- **Ford-Fulkerson:** mientras haya camino $s \rightsquigarrow t$ en G_f , aumentar el flujo en el cuello de botella del camino. Termina (con capacidades enteras) porque cada iteración suma ≥ 1 al valor del flujo, acotado por cualquier corte.
- **Teorema Max-Flow = Min-Cut:** $\max_f v(f) = \min_{\{A,B\}} c(A,B)$. Se cumple exactamente cuando ya no queda camino aumentante; el conjunto A = alcanzables desde s en G_f final da directamente el corte mínimo.
- **Complejidad:** genérico $O(E \cdot v(f^*))$ — **no polinomial en el tamaño de entrada en general.** Edmonds-Karp (BFS) $O(VE^2)$ — **sí polinomial**, porque acota iteraciones en $O(VE)$ sin depender de las capacidades.
- **Matching bipartito $\rightarrow$ Max-Flow:** $s \rightarrow L$ (cap. 1), $L \rightarrow R$ según aristas originales (cap. 1), $R \rightarrow t$ (cap. 1). El valor del flujo máximo = tamaño del emparejamiento máximo.
- **Trampas:** (1) olvidar aristas hacia atrás, (2) camino no realmente aumentante (capacidad residual 0 en algún tramo), (3) corte mínimo = suma de capacidades de un conjunto de aristas, no una sola arista, (4) genérico no es polinomial en el tamaño de entrada, (5) conservación no aplica en s/t .